Paint Color Detection

Course:	COSI 149B - Practical Machine Learning with Big Data
Association:	Brandeis University, Waltham, MA, USA
Instructor:	Prof. Pengyu Hong
Authors:	Zepeng Hu
	Nhi Le
	Yurim Lee
Date:	March 23, 2025
Repository:	https://github.com/halannhile/paint-color-detection

Abstract

This project addresses the challenge of accurate color reproduction across varying lighting conditions and camera settings. We developed a color calibration system that uses a physical reference sheet with distinct shapes (red circle, blue pentagon, green triangle) to calibrate colors in photographs. By fine-tuning a YOLOv8 model for object detection and experimenting with various different models, from classical ML models to deep learning models, we achieved high accuracy in determining true RGB values from observed colors. Our best system achieves an R^2 score of 0.924 and a mean Delta E of 0.0168, demonstrating its effectiveness for detecting paint color, proving its usefulness for applications in interior design, digital media, and color matching. The solution is currently been deployed as a web application, making it accessible to users without specialized knowledge or equipment.

Contents

1 Task Definition

When working on interior design or fabric selection, users often need to find colors that closely match what they already have. However, environmental factors (lighting conditions, brightness, shades) and device factors (camera settings, sensor resolution) frequently cause discrepancies between the photographed color and the purchased color. Additionally, at paint or fabric companies, it can be difficult to identify the desired color among numerous options, and visiting in person to get assistance from staff is not always practical. To address these challenges, we aim to develop a system where users can upload photos of their desired space colors and receive accurate RGB and HEX values that most closely match their target. This effectively implements an offline color picker.

Our technical approach involves developing a physical color calibration tool to compensate for environmental variables. Specifically, we use a standard reference sheet printed with a red circle, blue pentagon, and green triangle. When these shapes are photographed alongside the target area, we analyze the relative difference between the detected RGB values in the image and the actual RGB values set during printing. Based on this difference, we can accurately extract the true color values of the target area. This project consists of two phases:

1. Extraction Phase: Extract RGB values from the target color area and reference shapes in the image
2. Prediction Phase: Analyze the color differences of the reference shapes to predict the actual color values of the target area

This system will enable users to conveniently and accurately obtain desired color information anytime and anywhere.

2 Data Collection & Preparation

2.1 Annotating data and finetune YOLOv8 model for landmark detection

Since the YOLO model family, specifically YOLOv8 version which we decided to use in this case, is only good at detecting some predefined shapes such as 'person', 'car', 'dog', etc., we had to finetune the YOLOv8 model with annotated data so that it can detect the calibration tool and the landmarks (circle, triangle, pentagon, square) in our collected images. To this end, we decided to hand-annotate $\sim$130 images by drawing bounding boxes for 5 classes: 'color-calibration-tool', 'circle', 'triangle', 'pentagon', 'paint-color' and split them into train-val-test sets in 80-10-10 ratio to feed into the YOLOv8 model for fine-tuning. We used an open-source annotation website called Roboflow to perform this task.

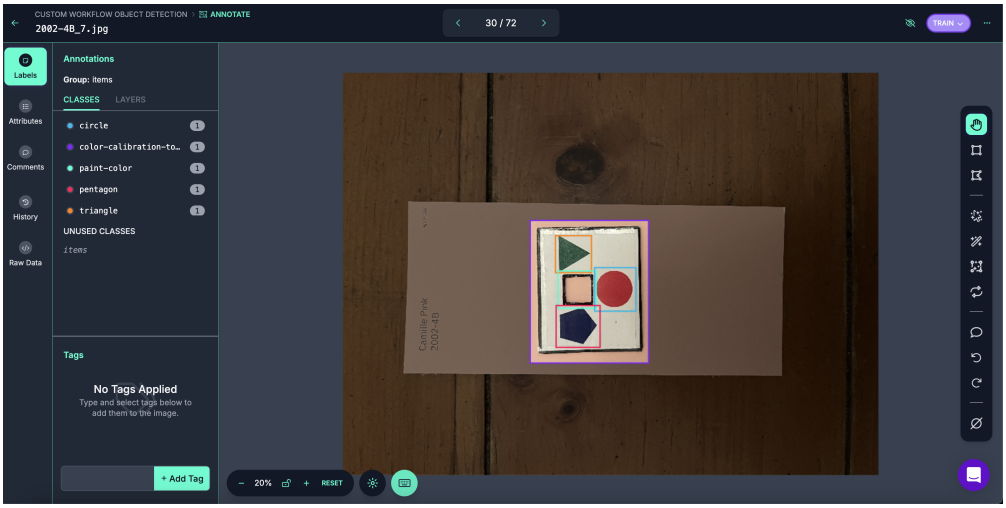

Figure 1: Screenshot of the Roboflow annotation interface

After hand-annotation, for each image, we will be able to obtain a corresponding .txt file describing the objects detected in that image, the class name of each object, as well as the coordinates of the object. For example, for the input image 1001-4A_1.txt, there are 5 objects (class names 0, 1, 2, 3, 4) with the following coordinates of the object's bounding box:

data > annotations > 1001-4A_1.txt

```
3 0.5816583333333334 0.5260125 0.3494333333333333 0.22101874999999999
0 0.5768833333333333 0.47430625 0.10154166666666666 0.07576875
4 0.4800083333333333 0.55530625 0.10049166666666667 0.06853125
1 0.5800333333333333 0.5494812499999999 0.09263333333333333 0.06638125
2 0.6777166666666666 0.54881875 0.10290833333333332 0.082175
```

Figure 2: Example of annotation format

This is the format required to fine-tune YOLOv8. We can then use the trained model to detect objects in an unseen image with high confidence (¿ 0.9 confidence):

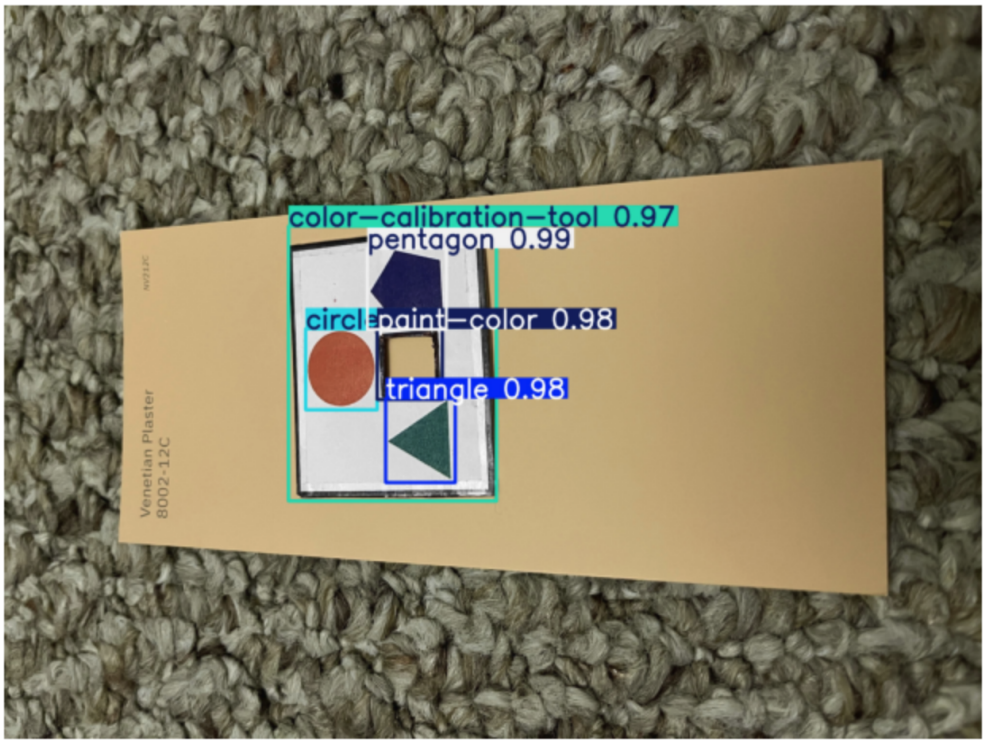

Figure 3: Example of object detection using the fine-tuned YOLOv8 model

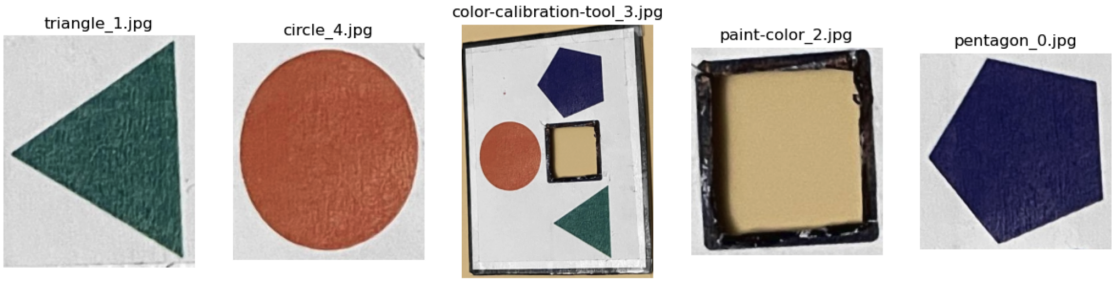

Figure 4: Detected objects using the fine-tuned YOLOv8 model

Once the objects are detected, we can sample a small region in the center of each object, get the average RGB codes of that region, and use this as the RGB codes of the object.

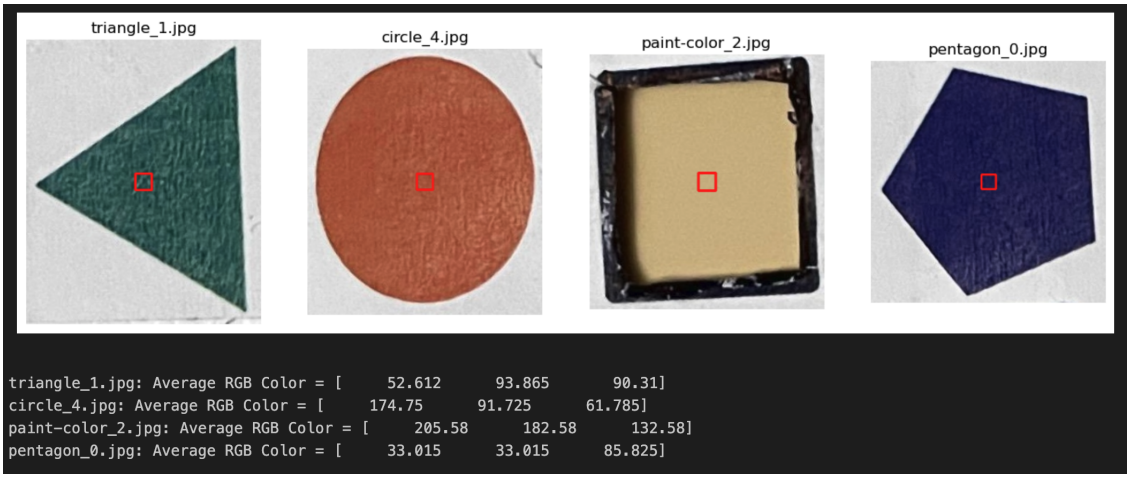

Figure 5: Sampling RGB values from the center of detected objects

To confirm that the average RGB codes we extract are correct, we can reconstruct the colors based on these RGB codes and confirm that they are indeed the colors in the detected objects:

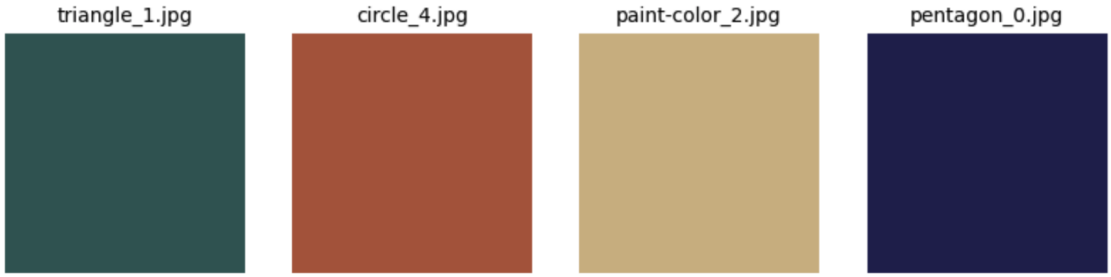

Figure 6: Reconstruction of colors from extracted RGB values

The trained YOLOv8 model is used in different components of our project, from creating the train-val-test datasets, to extracting the observed RGB codes of unseen images during real-time inference phase.

The train-val-test datasets are created from 230 collected images of different paint colors, under different lighting conditions and photo angles. Each row in our datasets corresponds to an image, with these columns:

- color_code: the code of the paint color (e.g., 8002-12C).
- file_name: the name of the image file (e.g., 8002-12C_2, where the suffix _2 indicates that this is the second image out of all the images taken of this paint color under different conditions).
- true_R, true_G, true_B: the true RGB codes of the paint color (this can be obtained from the paint manufacturer's website, e.g., for 8002-12C, we can find its true RGB code from this site). This acts as the ground truth for all our models.

- observed_R, observed_G, observed_B: the RGB codes of the paint color in the photo.
- red_R, red_G, red_B: the RGB codes of the red circle in the photo (since we know that the true RGB codes for primary red is (255, 0, 0), it is redundant to include these columns in the dataset, same thing goes for the true blue and true green landmarks).
- green_R, green_G, green_B: the RGB codes of the green triangle in the photo (the true RGB codes of primary green color is (0, 255, 0)).
- blue_R, blue_G, blue_B: the RGB codes of the blue pentagon in the photo (the true RGB codes of primary blue color is (0, 0, 255)).

Here is a snippet of what each of train-val-test sets looks like:

color_code	file_name	true_R	true_G	true_B	observed_R	observed_G	observed_B	red_R	red_G	red_B	green_R	green_G	green_B	blue_R	blue_G	blue_B
1004-4C	1004-4C-4	238	215	226	158.0	155.0	149.0	156	69	56	114	140	117	86.0	93.0	112.0
1004-4B	1004-4B_5	234	200	217	151.0	140.0	158.0	160	99	122	109	123	127	76.0	82.0	111.0
2002-4B	2002-4B_7	239	184	184	177.0	147.0	129.0	184	92	91	152	163	145	110.0	110.0	119.0
8002-12B	8002-12B_6	236	187	168	178.0	161.0	135.0	200	108	99	134	160	140	116.0	126.0	150.0
7003-10	7003-10-2	232	216	189	159.0	152.0	137.0	153	63	59	115	136	116	86.0	90.0	123.0
7002-10	7002-10-2	244	230	210	165.0	162.0	153.0	150	65	65	119	140	125	94.0	99.0	129.0
7001-19	7001-19-4	228	213	207	158.0	156.0	149.0	166	76	71	126	147	131	84.0	88.0	116.0

Figure 7: Snippet of the dataset

2.2 Prediction Model Data Information

For our color calibration system, we collected a comprehensive dataset consisting of 230 images across 49 distinct color codes. Each color code was photographed approximately 5 times under varying lighting conditions and camera settings to ensure robustness in our model development.

All images were captured with our standardized reference sheet containing pre-defined shapes: a red circle, a blue pentagon, and a green triangle. These reference shapes serve as calibration points for our color prediction model.

The extracted dataset includes the following columns: 'color_code': Identifier for the target color, 'file_name': Image source identifier, 'true_R', 'true_G', 'true_B': The actual RGB values of the target color, 'observed_R', 'observed_G', 'observed_B': RGB values extracted from the target area in the photo, 'red_R', 'red_G', 'red_B': RGB values of the red circle reference, 'green_R', 'green_G', 'green_B': RGB values of the green triangle reference, 'blue_R', 'blue_G', 'blue_B': RGB values of the blue pentagon reference.

Our prediction model aims to accurately estimate the true RGB values (true_R, true_G, true_B) based on the observed values from the target area and the reference shapes in the photographs. Due to the limited dataset size, we divided our data into training, validation, and test sets using a 9:1:1 ratio to ensure sufficient training data while maintaining appropriate validation and testing capabilities.

Importantly, we ensured that all samples with the same color_code were assigned to the same partition (training, validation, or test). This stratification approach prevents data leakage and provides a more realistic evaluation of the model's ability to generalize to unseen colors rather than just unseen lighting conditions of familiar colors.

3 Models

3.1 Auto-Encoder

We chose an autoencoder for our color prediction task based on its ability to compress and reconstruct information while preserving essential features. This characteristic seemed particularly well-suited to our challenge of color calibration across varying conditions. Through the encoding-decoding process, we hypothesized that the model could effectively learn to account for differences in camera settings and lighting conditions. Additionally, autoencoders are known for their robustness to noise - we conceptualized the variations in lighting conditions and camera differences as a form of noise affecting color values, which the autoencoder could potentially filter out.

For our development environment, we utilized Colaboratory notebooks. The model architecture consists of an input layer, encoder, latent space, decoder, and output layer. The input dimension is 12 (RGB values from 4 colors), while the output dimension is 3 (predicted RGB values). To optimize model performance, we implemented a composite loss function combining MSE loss, weighted RGB loss, and color similarity loss. The training process employed an Adam optimizer with a ReduceLROnPlateau scheduler, while dropout and early stopping techniques were applied to prevent overfitting.

3.1.1 K-Means Hyperparameter Optimization Approach

To determine the optimal hyperparameters for our model, we conducted a series of experiments with various settings. Given our limited dataset, we employed a K-fold cross-validation approach to maximize data utilization. We reserved 20% of the total dataset as a test set, and with the remaining data, we implemented 5-fold cross-validation to divide between training and validation sets.

Table 1: Performance variation by encoder and decoder layers

enc	dec	RMSE	R^2	Delta E
3	2	14.4008	0.6546	8.3269
3	3	15.0071	0.6380	8.5174
2	2	14.0895	0.6572	8.2066
2	3	14.6559	0.6485	8.1804

For our K-fold experiments, we maintained consistent baseline parameters: learning rate (0.001), MSE loss weight (0.2), weighted RGB loss weight (0.5), cosine similarity loss weight (0.3), latent space dimension (256), and dropout rate (0.1). We specifically focused on evaluating different configurations of encoder and decoder layers.

Interestingly, our results showed that a simpler architecture with both encoder and decoder at 2 layers each produced the best performance. Contrary to our initial hypothesis, the more complex setting with 3 layers for both encoder and decoder actually recorded the highest Delta E values, indicating poorer color reproduction accuracy. This finding suggests

that for our specific color calibration task, a more streamlined network architecture provided better generalization capabilities than deeper alternatives.

Table 2: Performance variation by Dropout & Latent Space

Latent Space	dropout	RMSE	R^2	Delta E
256	0.2	15.0405	0.6311	8.9252
512	0.1	14.9008	0.6306	8.4477
512	0.1	15.4715	0.6150	8.8856
128	0.1	14.8572	0.6369	8.5698

We also conducted experiments by varying the latent space dimension and dropout rate. When we reduced the latent space to 128, we observed a noticeable decrease in performance. Similarly, keeping the latent space at 256 while increasing the dropout rate to 0.2 also resulted in diminished performance. We attribute these results to the relatively small number of parameters being used for training.

To address this, we attempted to increase the latent space dimension. However, this approach led to what appeared to be overfitting, suggesting that simply increasing the number of parameters does not necessarily make the model more robust. This reinforces the fundamental principle in machine learning that finding the appropriate parameter balance is crucial rather than arbitrarily increasing model complexity.

These experiments highlight the importance of careful hyperparameter tuning in machine learning tasks. The optimal configuration requires balancing between underfitting (too few parameters) and overfitting (too many parameters) to achieve a model that generalizes well to unseen data while capturing the essential relationships in the training set.

Table 3: Performance variation based on loss function weights

MSE Loss	weighted RGB loss	cosine similarity loss	RMSE	R^2	Delta E
1	0	0	14.7433	0.6492	8.5367
0	1	0	15.2534	0.6310	9.4661
0	0	1	45.8031	-1.9284	18.2003
0.5	0.3	0.2	14.3091	0.6545	8.3182
0.3	0.5	0.2	14.4290	0.6481	8.3994

Next, we experimented with different weight configurations for our optimizer's composite loss function. Among the three components—MSE loss, weighted RGB loss, and cosine similarity loss—using cosine similarity loss alone yielded the poorest results. Conversely, using MSE loss by itself produced the best individual performance.

Interestingly, when combining all three loss components, we discovered that assigning higher weight to the weighted RGB loss, followed by cosine similarity and then MSE loss, resulted in superior overall performance compared to using MSE alone. Based on these experiments, we finalized our loss function weights as follows: MSE loss at 0.2, weighted RGB loss at 0.5, and cosine similarity loss at 0.3.

This optimal weighting indicates that while MSE provides a solid foundation for color prediction, incorporating domain-specific losses that consider both the relative importance of RGB channels and perceptual color similarity contributes significantly to the model's ability to accurately restore true colors from observed values.

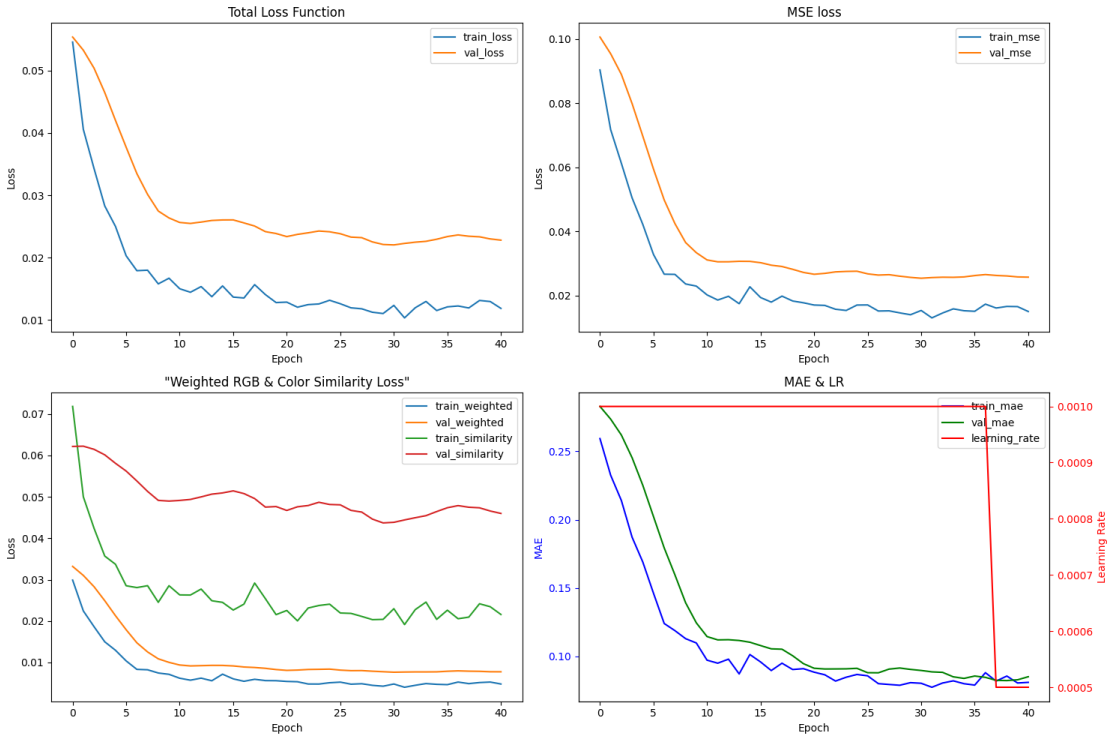

Figure 8: Loss and learning rate changes during training in the optimal fold

Based on our experimental results, we concluded that the optimal hyperparameter configuration was: MSE loss weight (0.2), weighted RGB loss weight (0.5), cosine similarity loss weight (0.3), latent space dimension (256), and dropout rate (0.1). This combination provided the best balance between model complexity and performance on our color calibration task.

The graph above visualizes the training metrics from the best-performing fold(fold 2) in our k-fold cross-validation using these optimal parameters. It displays the progression of MSE loss, weighted RGB loss, cosine similarity loss, total loss, and MAE-learning rate throughout the training process.

As training progressed, we observed that all loss components—MSE, weighted RGB, and color similarity—decreased over time. However, the color cosine similarity loss showed notably different reduction patterns between training and validation sets compared to other loss functions. This discrepancy suggests that the perceptual color similarity metric behaves differently from traditional numerical error metrics and may capture aspects of color relationships that are more challenging to generalize across unseen data.

3.1.2 Final Model Training and Evaluation

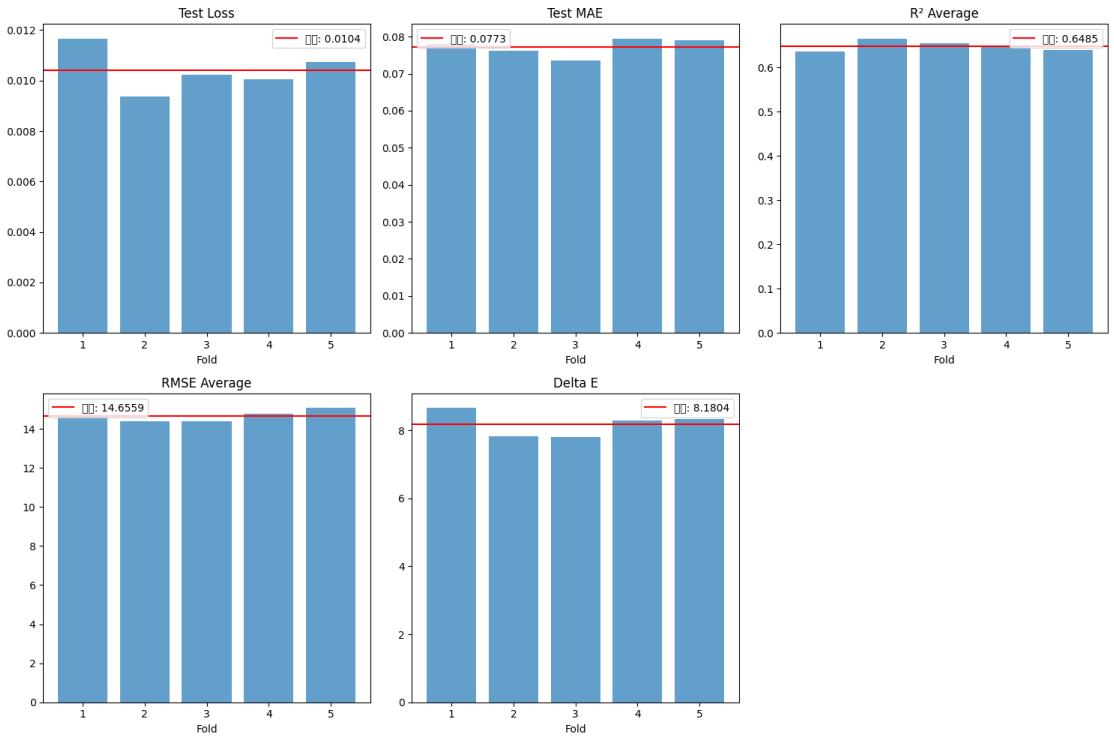

Figure 9: Comparison of loss across folds with optimal hyperparameters

Table 4: Performance variation based on learning rates

LR	RMSE	R^2	Mean_Delta E	Median_DeltaE
0.005	13.6947	0.8337	7.3536	4.5287
0.001	14.6954	0.8033	8.0298	5.2367
0.01	13.6980	4.7856	7.9700	4.7856

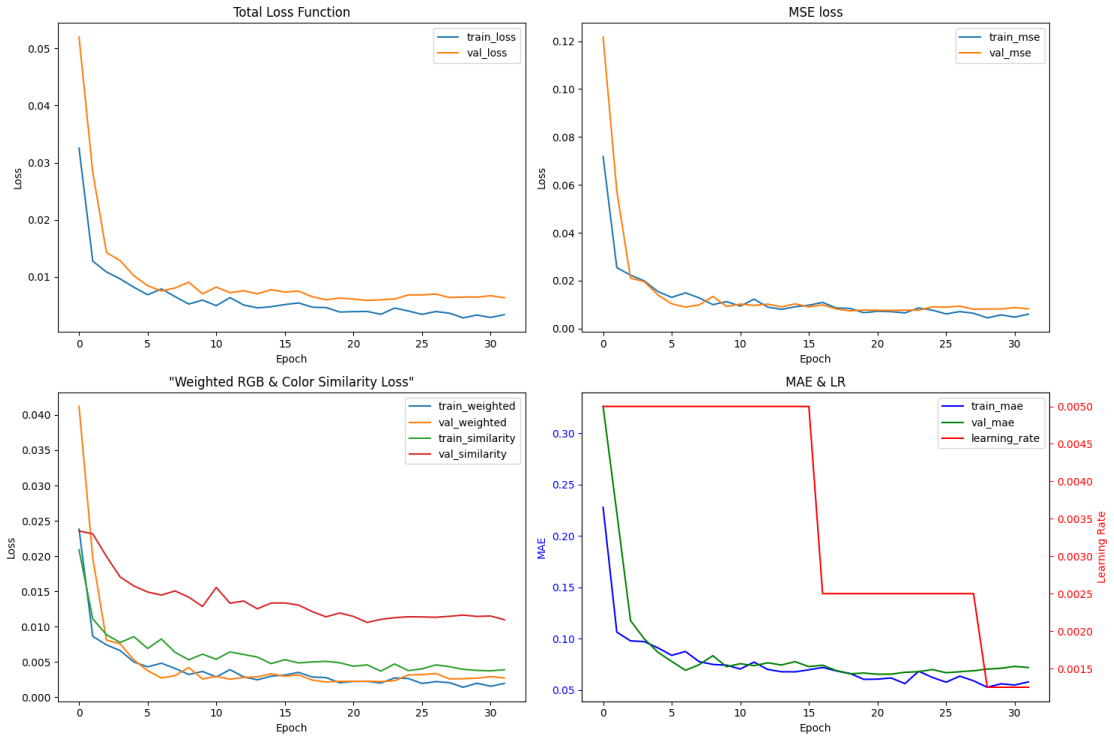

Figure 10: Loss and learning rate changes during training in the final model

After determining the optimal hyperparameters, we proceeded to train and evaluate our final model using an expanded dataset split into train, validation, and test sets at an 8:1:1 ratio. During this process, we also conducted additional experiments regarding the learning rate. While we had used a learning rate of 0.001 during our k-fold experiments, we discovered that a learning rate of 0.005 yielded superior performance, which we ultimately adopted for our final model.

Based on the observed progression of the total loss, we determined that 32 epochs was appropriate for the training duration. This provided sufficient iterations for the model to converge while avoiding the potential for overfitting that might occur with extended training periods.

Figure 11: Comparison of Color Restoration Results (Sorted by Smallest Color Difference)

The image presents a comparison between the colors predicted by our final autoencoder model on the test dataset and their actual RGB values. Focusing on the top 11 most accurate predictions, we can observe that while not achieving perfect matches, the model demonstrates impressive accuracy in color measurement and restoration. The similarities between the predicted and actual colors validate our approach's effectiveness for this color calibration task.

However, the subtle differences that remain highlight the potential for further improvement. These discrepancies suggest that additional training data or alternative learning approaches might be beneficial for achieving even greater precision in color prediction. This

insight provides valuable direction for future refinements to the model and methodology.

3.1.3 RGB-to-Valspar Color Matching with Gemini API

For user convenience, we developed an agent that predicts RGB values using our model and converts them to HEX format, then leverages the Gemini API to extract the three most similar colors from our collected VALSPAR dataset. The agent provides comprehensive information including RGB values, similarity scores, color codes, and color values. In our testing with the gemini-2.0-flash model, we were able to create a streamlined process that bridges the gap between digital color analysis and real-world paint selection, allowing users to quickly identify Valspar paint options that closely match their desired colors without manual conversion or searching.

```
prompt = f"""
As a color expert specializing in the Valspar paint catalog:

I need to identify the 3 closest matches in the Valspar paint collection to a specific color:
- RGB: ({rgb_values[0]}, {rgb_values[1]}, {rgb_values[2]})
- HEX: {hex_color}

Please analyze this color and identify the closest Valspar paint colors by comparing RGB values and visual similarity.

For the 3 closest matching Valspar colors, provide:
1. Color Name
2. Color Code/Number
3. RGB Values (provide exact numbers)
4. HEX Code
5. Brief Description (texture, mood, common uses)
6. Similarity Score (1-10, with 10 being identical)

Format each match clearly with all details in a consistent structure.

Important: Only include real, currently available Valspar colors with accurate information from their catalog. Do not invent colors or data.
```

Figure 12: Prompt on Gemini

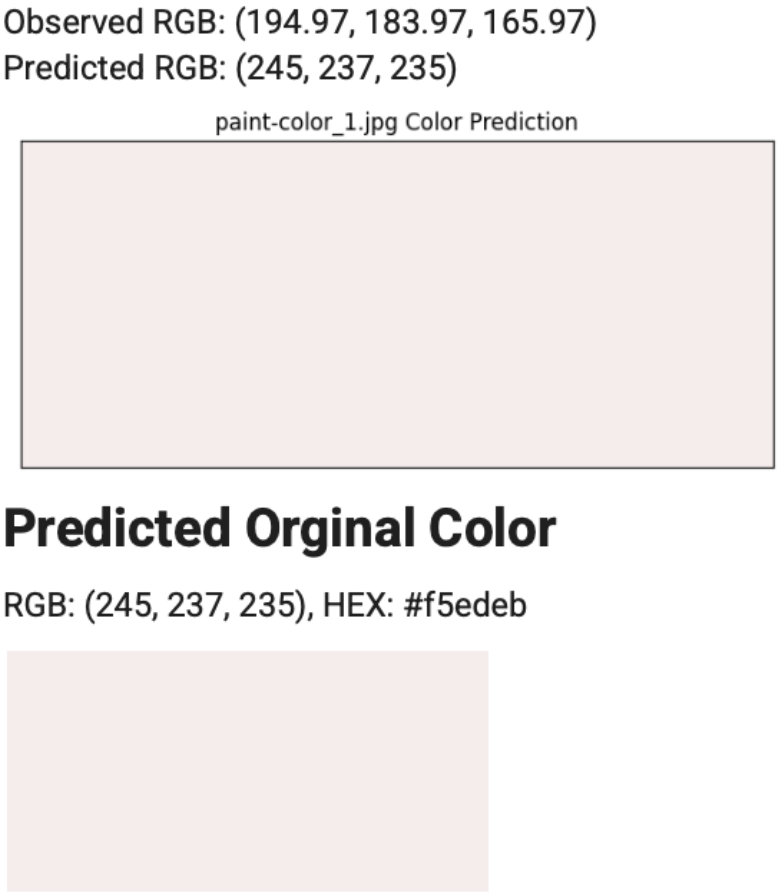

Figure 13: Color prediction visualization

Valspar Color Matching Result

Okay, here are the 3 closest Valspar paint colors I can identify to the specified RGB (245, 237, 235) / HEX #f5edeb, based on a thorough review of the Valspar catalog. Matching colors is a multifaceted process, I will consider both the mathematical RGB delta and subjective visual similarity. The exact RGB values are used in the scoring process to rank colors by closeness, with subjective interpretation applied only when the mathematical results are close.

Here are the top 3:

Match 1:
1. **Color Name:** Winter Bliss
2. **Color Code/Number:** V112-1
3. **RGB Values:** 246, 238, 234
4. **HEX Code:** #f6eeea
5. **Brief Description:** A soft, warm white with subtle peach undertones. Has a calming and airy feel. Often used in bedrooms, nurseries, and living rooms to create a cozy atmosphere. Usually available in matte or eggshell finish for a smooth appearance.
6. **Similarity Score:** 9/10

Match 2:
1. **Color Name:** Ivory Lace
2. **Color Code/Number:** 7002-16
3. **RGB Values:** 245, 235, 232
4. **HEX Code:** #f5ebe8
5. **Brief Description:** A creamy, off-white that leans towards a classic ivory. Possesses a comforting and inviting quality. Frequently used for trim, cabinets, and walls in traditional and transitional spaces. Looks good in a satin or semi-gloss for greater durability.
6. **Similarity Score:** 8/10

Match 3:
1. **Color Name:** Frosted Linen
2. **Color Code/Number:** V108-1
3. **RGB Values:** 241, 234, 231
4. **HEX Code:** #f1eae7
5. **Brief Description:** This is a very light, slightly cool off-white with barely perceptible grey undertones. Creates a serene and neutral backdrop. Commonly used in modern and minimalist interiors, suitable for walls and ceilings. Matte and eggshell finishes are common.
6. **Similarity Score:** 7/10

Figure 14: Valspar Color Matching Result

3.2 Classical ML models: Linear Regression, Random Forest, SVM, XGBoost, LightGBM, AdaBoost

In this color calibration task, the objective was to predict the true RGB values of target input colors using various machine learning models, based on the observed input from a color watch. A range of regression models were evaluated, including Linear Regression, Random Forest, Support Vector Machine (SVM), XGBoost, LightGBM, and AdaBoost. Among all models tested, AdaBoost achieved the best overall performance with an R^2 score of 0.2788, RMSE of 0.1112, and a mean ΔE of 0.0215, indicating strong agreement with the ground truth. LightGBM and SVM also performed well, with R^2 scores of 0.2681 and 0.2646 respectively, and comparable RMSE and ΔE metrics. In contrast, the Linear Regression model showed significantly lower performance ($R^2 = 0.0507$), highlighting the limitations of linear approaches for capturing the complex relationship between input and target colors. Notably, Random

Forest yielded a relatively low mean ΔE of 0.0195 and the lowest median ΔE of 0.0130, making it highly effective in minimizing perceptual color errors. Overall, ensemble-based methods—particularly AdaBoost and Random Forest—demonstrated superior performance in accurately predicting calibrated RGB values, which is essential for consistent and precise color reproduction in real-world applications.

Table 5: Model Performance Metrics

Model	R^2 Score	RMSE	MAPE (%)	Mean ΔE	Median ΔE
Linear Regression	0.051	0.1265	18.97	0.0406	0.0323
Random Forest	0.243	0.1136	9.53	0.0195	0.0130
SVM	0.265	0.1132	10.56	0.0225	0.0173
XGBoost	0.196	0.1173	10.54	0.0296	0.0111
LightGBM	0.268	0.1129	8.89	0.0201	0.0118
AdaBoost	0.279	0.1112	9.82	0.0215	0.0141

3.3 Ensemble of Neural Networks and Tree-based Models

We also developed another system that combines neural networks and tree-based models in an ensemble architecture to predict the true RGB values from observed colors. The main motivation between ensembling NN and tree-based models is to leverage the strengths of deep learning models for complex pattern recognition, and of traditional ML models to model robust feature relationships, achieving impressive color prediction accuracy across varied lighting conditions.

3.3.1 Initial Data Preparation

- Input data: RGB values of observed paint color and calibration shapes (12 values total)
- Normalization: All RGB values are scaled from [0-255] to [0-1] range
- Missing value handling: NaN values are replaced with column means or zeros
- Outlier detection: Values outside the 1st-99th percentile range are clipped to prevent extreme values from influencing the model

3.3.2 Feature Engineering

Instead of relying solely on the observed RGB codes as features for the model, we engineered several new features. These features allowed us to achieve substantial performance improvements over traditional ML models.

- Basic features:
  - RGB differences: Differences between observed color and each calibration color

  - RGB ratios: Ratios between observed color and each calibration color

- Advanced color features:
  - Luminance calculations: Weighted combinations of RGB channels (Luminance = 0.299 * Red + 0.587 * Green + 0.114 * Blue). These coefficients are derived from the ITU-R BT.601 standard, which was used for standard-definition television. The idea is that not all colors contribute equally to perceived brightness: Red is less sensitive to the human eye → 0.299, Green is the most sensitive → 0.587, Blue is the least sensitive → 0.114. So these values reflect human visual perception, not the physical energy of light.
  - Color statistics: min, max and range values across channels.
  - Channel interactions: Products between color channels (R*G, R*B, G*B)

- Color space transformations:
  - LAB color space: Conversion from RGB to perceptually uniform LAB space.
  - HSV color space: Alternative representation capturing the hue, saturation and value. This is useful for color clustering.

- Color correction matrices:
  - Physics-based correction: Calculating transformation matrices using calibration colors. This is useful for several reasons: 1. Cameras see colors differently under warm (yellow) vs. cool (blue) lighting. This correction accounts for that, 2. Some cameras naturally capture reds/greens/blues more strongly. This calibrates that out.

Details on how these features were engineered can be found in our codebase.

3.3.3 Data augmentation

To improve model robustness, several augmentation techniques are applied:

- Global intensity shifts: Scaling all RGB values by factors (0.95-1.05)
- Channel-specific shifts: Selectively scaling R, G, or B channels
- Color temperature shifts: Modifying the R/B ratio to simulate warm/cool lighting
- Random noise: Adding small Gaussian noise to RGB values

3.3.4 Neural Network Architecture

1. Input Layer: Dense layer with BatchNorm, LeakyReLU(0.1), Dropout(0.2)
2. Hidden Layers:
   - 4 Residual blocks with attention mechanisms

- Each block: Linear → BatchNorm → LeakyReLU → Linear → BatchNorm → Attention → Skip connection

3. Parallel Processing Paths:
   - Luminance path: Processes brightness information
   - Chrominance path: Processes color information
4. Output Layer: Three separate heads for R, G, B channels
5. Attention Mechanism
6. Color transformation layer: A specialized layer that explicitly models the physical color transformation

3.3.5 Neural Network Training Process

- Optimization: AdamW optimizer with weight decay (1e-5)
- Learning rate: OneCycleLR scheduler starting at 0.0005 with 30% warmup
- Loss function: Custom perceptual loss combining MSE, L1, and luminance components
- Regularization: Gradient clipping, dropout, and batch normalization
- Early stopping: Based on validation loss with patience of 50 epochs
- Batch size: 24 samples
- Training epochs: 100 epochs with early stopping

3.3.6 Tree-based Models

Besides the main component, which is the neural network, the ensemble also includes four different tree-based models, each with specialize hyperparameters:

- Gradient boosted regressor: n_estimators=500, learning_rate=0.02, max_depth=5, subsample=0.7, min_samples_split=3, random_state=42
- Random forest regressor: n_estimators=500, max_depth=12, min_samples_split=2, max_features='sqrt', random_state=42
- XGBoost regressor: n_estimators=500, learning_rate=0.02, max_depth=6, subsample=0.7, colsample_bytree=0.7, random_state=42
- LightGBM regressor: n_estimators=500, learning_rate=0.02, max_depth=5, num_leaves=31, subsample=0.7, random_state=42

3.3.7 Ensemble Architecture

The final prediction combines the neural network and tree-based models using a weighted average: Neural Network: 0.50, Gradient Boosting: 0.15, Random Forest: 0.10, XGBoost: 0.20, LightGBM: 0.05.

Weight Optimization: The weights are derived through a two-stage process:

1. Grid search on validation set to find weights that minimize Delta E
2. Fine-tuning with focus on perceptual accuracy rather than just statistical metrics

3.3.8 Post-Processing Pipeline

After making predictions, we implemented a perceptual post-processing step further refines the results:

- Saturation adjustment: Matching prediction saturation to target saturation levels
- Brightness adjustment: Aligning overall brightness with expected values
- Dominant channel correction: Ensuring the correct color channel dominates

Details on how these post-processing steps are implemented can be found in our codebase.

3.3.9 Evaluation metrics

Similarly, for the ensemble model, the system is evaluated using multiple metrics: R-squared, RMSE, MAPE, Mean Delta-E, Median Delta-E.

4 Experimental Results

Table 6: Model Performance Metrics (**best**, second best)

Model	R^2 Score	RMSE	MAPE (%)	Mean ΔE	Median ΔE
Linear Regression	0.051	0.1265	18.97	0.0406	0.0323
Random Forest	0.243	0.1136	9.53	0.0195	0.0130
SVM	0.265	0.1132	10.56	0.0225	0.0173
XGBoost	0.196	0.1173	10.54	0.0296	**0.0111**
LightGBM	0.268	0.1129	8.89	0.0201	0.0118
AdaBoost	0.279	0.1112	9.82	0.0215	0.0141
Autoencoder (normalized)	0.920	0.0372	4.05	0.0244	0.0174
NN-Tree Ensemble	**0.924**	**0.0355**	**3.61**	**0.0168**	**0.0111**

This table summarizes the performance metrics of all models developed for the color calibration task. The Neural Networks and Tree-Based Models Ensemble achieved the best overall performance with an R^2 score of 0.924, RMSE of 0.0355, and the lowest MAPE of 3.61%. It also demonstrated superior color accuracy with a mean ΔE of 0.0168 and median ΔE of 0.0111, which represents the best perceptual color matching among all the models tested.

Comparing the traditional ML approaches to the deep learning and ensemble methods, we observe that while AdaBoost was the best performer among the classical ML models, both the Autoencoder and the NN-Tree Ensemble significantly outperformed all classical approaches. This suggests that the complex relationships between observed colors and their true values are better captured by deep learning architectures and hybrid approaches that combine the strengths of different model types.

5 Project Deployment

This deployment approach makes our color calibration technology accessible to users without requiring specialized knowledge or equipment. The web-based interface allows the user to upload an image. The trained YOLOv8 will then detect the objects and extract the observed RGB codes. The user will then be able to choose a model to run (the options include all of our models: Neural Network & Tree-based Models Ensemble, Classical ML models, Autoencoder). They will then be able to see the color and RGB codes predicted by the model, compared against the true color. Due to the limited time of the project, this is still a work in progress, but we will release a fully working version in the next iteration of our work.

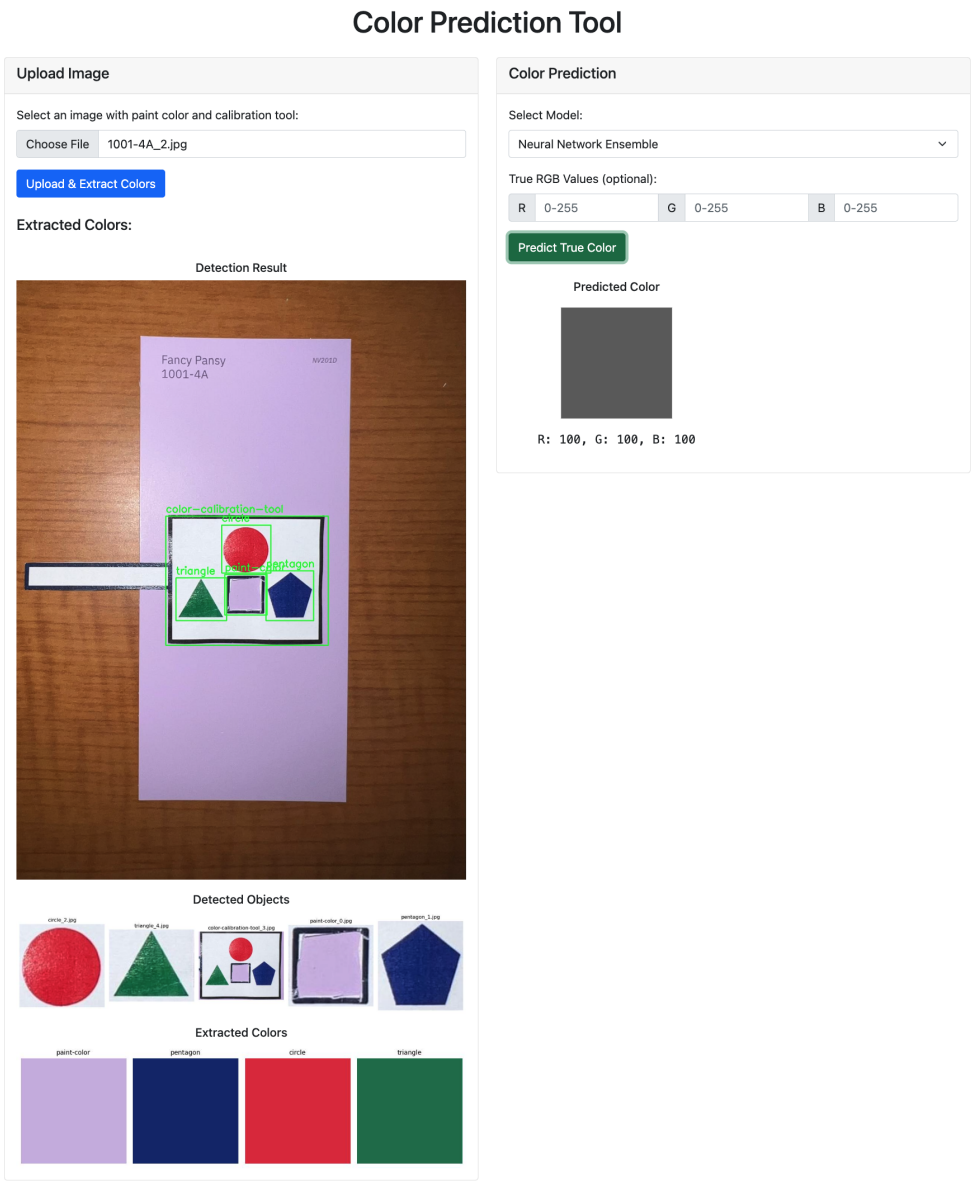

Figure 15: Demo of project website (work in progress)

6 Future Work

1. Enhanced YOLOv8 Training for Extreme Conditions: A key improvement would be expanding the training data for the YOLOv8 object detection model to include images captured under extremely poor lighting conditions (very dark, overly bright, or highly uneven illumination) and from extreme angles. Currently, while our model performs well under standard conditions, its accuracy decreases in edge cases. By collecting and annotating a more diverse dataset with these challenging scenarios, we could develop a more robust object detection system that can reliably extract RGB codes even in suboptimal photography conditions, significantly increasing the real-world usability of our application.

2. Integration with Color Catalogs Our current system predicts true RGB values but could be expanded to integrate with professional color catalog systems such as Pantone, RAL, or specific paint manufacturer databases. This integration would allow the system to not only provide RGB values but also match colors to the closest commercially available options across multiple manufacturers, making it immediately actionable for consumers and professionals. This would bridge the gap between digital color representation and physical product selection.

3. Advanced Neural Network Architectures While our current neural network ensemble yields impressive results, exploring more advanced architectures could further improve prediction accuracy. Specifically, implementing transformer-based models with self-attention mechanisms across multiple color channels could better capture the complex relationships between observed and true colors. Additionally, experimenting with vision transformers (ViT) for the feature extraction phase might improve the system's ability to understand context and spatial relationships within the image, potentially leading to more accurate color predictions.

4. Adaptive Learning System Implementing an adaptive learning system that continuously improves based on user feedback would enhance long-term accuracy. If users have access to ground truth color values (through professional color measurement tools or reference samples), they could provide this data back to the system. This feedback loop would allow the models to adapt to specific use cases, environmental conditions, or camera characteristics, resulting in increasingly accurate predictions over time for individual users or user groups.

7 Conclusions

This project successfully developed and implemented a comprehensive color calibration system that addresses a common challenge in color reproduction across varying lighting conditions and camera settings. Through a systematic approach combining computer vision, machine learning, and color science, we created a solution that accurately predicts true color values from photographs.

The key achievements of our work include:

- Fine-tuning of a YOLOv8 object detection model to recognize the calibration tool and its components with high confidence (¿90%).
- Creation of multiple prediction models with progressive improvements in accuracy, culminating in the best-performing solution being an ensemble of neural networks and tree-based models.
- Achievement of notably good color prediction performance with an R^2 score of 0.924, RMSE of 0.0355, and mean Delta E of 0.0168.

- On-going deployment of a user-friendly web application that makes this technology accessible without specialized knowledge or equipment.

The comparative analysis of different prediction models revealed that while individual approaches like autoencoders or tree-based algorithms each have strengths, the hybrid ensemble architecture provides the most reliable color predictions across diverse conditions. This demonstrates the value of combining complementary approaches in solving complex computer vision problems.

Our perceptual post-processing pipeline proved essential for minimizing Delta E values, confirming that domain-specific knowledge of color science significantly enhances machine learning outcomes. The careful attention to feature engineering—including color space transformations, luminance calculations, and correction matrices—contributed substantially to the system's performance.

This project demonstrates the potential of combining traditional computer vision techniques with advanced machine learning approaches to solve practical, real-world problems. As discussed in our Future Works section, there remain exciting opportunities to further enhance and extend this system, but the current implementation already provides a valuable tool for color-accurate digital imaging.